2
Understanding Spring AI: Part 1

Understanding Spring AI is all about learning how to use Spring AI to interact with different AI models. An overview of Spring AI's features and functionality will help you utilize the capabilities of an AI model in the most effective manner possible.

To start with, it helps to have a fundamental understanding of Artificial Intelligence (AI) and its various disciplines. Then, you will learn about Spring's capabilities and role in AI. Through examples you will also learn how to use Spring AI's APIs effectively.

This chapter consists of two parts spread across two chapters, **Chapter 2** (this chapter) and **Chapter 3**. This chapter and **Chapter 3** are similar to reference guides and need not be read in any particular order.

We will cover the following topics in this chapter:

- A Brief Overview of AI
- Core APIs
- Moderation model API
- Embeddings model API
- Image model API
- Audio model API

Technical requirements

All examples are based on Java 17 and Spring AI 1.0. Additionally, all Spring AI applications are bootstrapped with Spring Boot 3.5.x.

While you always have a choice of AI models, depending on use case and requirements, most examples give you the option of running locally using Ollama, or, if you have an OpenAI or Google Cloud account, ChatGPT and Gemini, respectively. To run local AI models, download Ollama from https://ollama.com/download and install on your system of choice.

Additionally, when using Ollama, you will need to install a AI model from https://ollama.com/search. For example, install Meta's llama3.1 AI model using:

```
$ ollama run llama3.1
```

See llama3.1 page at https://ollama.com/library/llama3.1 for more details.

A brief overview of AI

The study and development of Artificial Intelligence (AI) has grown since its inception in 1956 to include many different disciplines. Artificial Intelligence is now a term used to generally classify all the various disciplines of AI, such as Machine Learning (ML) and Deep Learning (DL).

Each progressive discipline is a subset of the previous and can be loosely depicted as follows:

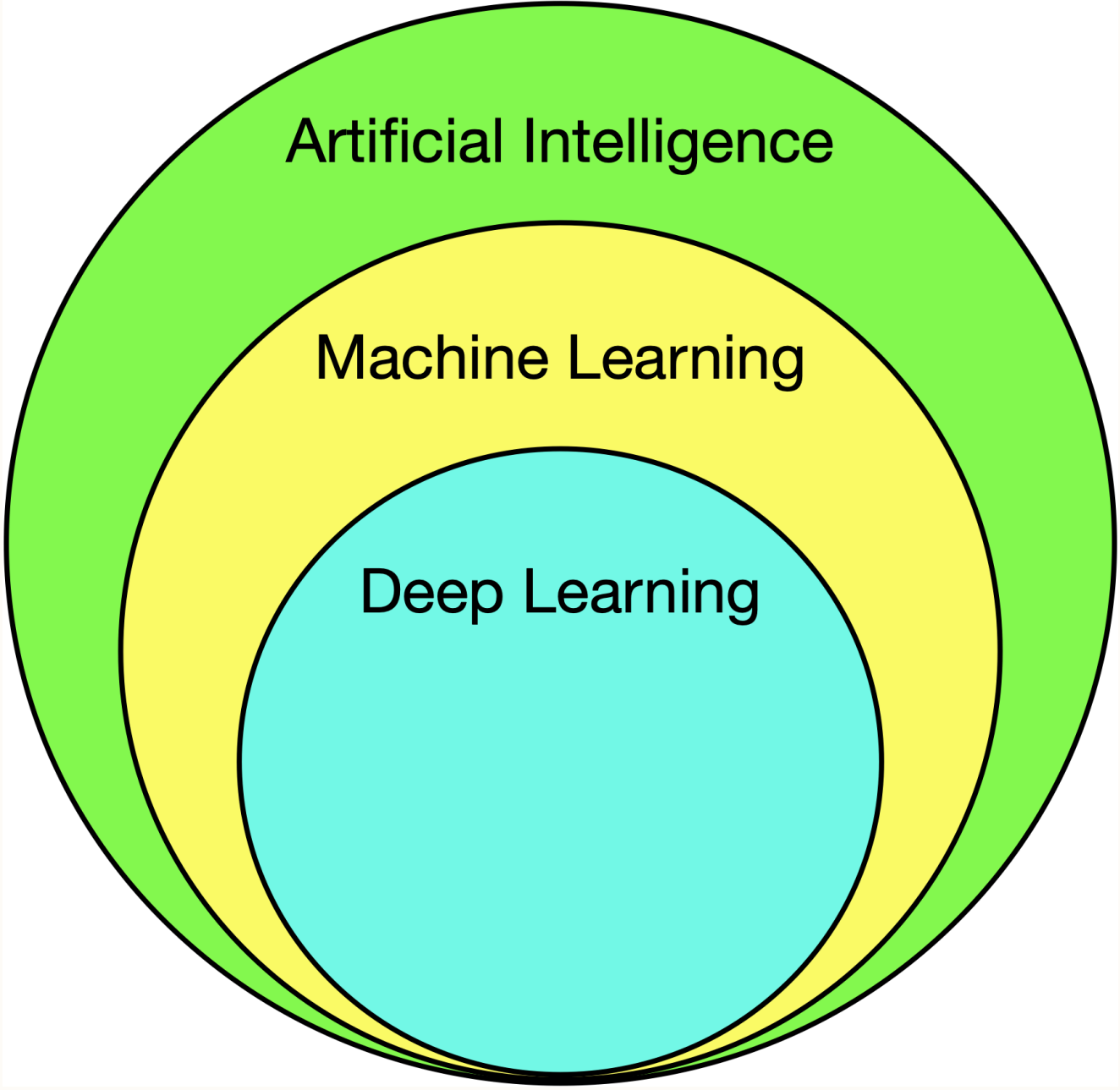


Figure 2.1: Disciplines of AI

Artificial Intelligence, referred to as AI, is a field of computer science that focuses on building computers and machines capable of reasoning, learning, and acting in a manner similar to humans. The goal is to have computers perform functions or tasks autonomously that would otherwise require human intelligence and supervision to perform. For instance, imagine a machine performing open-heart surgery and adapting to a patient's condition during the operation in real time.

AI, or computers in general, are better suited for performing complex and repetitive functions involving a lot of data to quickly identify patterns and form connections that are easily overlooked by humans. Although, like us, AI must be taught to learn these capabilities.

Machine Learning (ML), a subset of AI, focuses on training AI models with extremely large sets of labeled and unlabeled data. Labeled data is supervised, structured data, mapping a specific input to a precise or expected output, whereas unlabeled data is unsupervised, unstructured data, used in training models to recognize patterns.

Reinforcement learning is a familiar technique used to train an AI model as well. We see this in practice, as a form of feedback, when animals are fed a treat to reward the desired behavior. Learning by doing, reward/punishment, trial, and error, are all powerful techniques used as AI models are subjected to many iterations of extensive training and continuous refinement as new information becomes available.

At a fundamental level, AI models are nothing more than sophisticated computational systems and algorithms governed by mathematical functions, statistics, and a set of instructions.

When rooted in a particular data set, combined with statistical analysis, a model is guided toward decisions that are used to analyze, process, transform, and categorize data, look for patterns, make predictions, and even perform actions.

Not unlike glorified conditional statements or processing nodes in a binary decision tree, AI uses weights and probabilities in the decision-making process to generate the next most likely thing, whether it be the next word in a sentence, an image in a picture, or a sound in an audio file.

In an attempt to mimic both the structure and function of the human brain, scientists are using artificial neural networks to evolve ML into *Deep Learning* (DL). This is a rather ambitious effort to model cognitive functions computationally, in layers, and at scale, in order to enable an AI to continuously learn and adapt.

One of the more useful ways to classify an AI is by what an AI is capable of doing. Modern AI is mostly limited memory, reactive machines, or "narrow" intelligence that can only respond to external stimuli (events, motion, sounds, etc.). Like eyes, ears, nose, and mouth, sensors are the devices that an AI can use to perceive stimuli, or receive input, to gather information about its environment.

The ultimate expression of AI is to move ever closer to making decisions equal to that of humans, remembering experiences, interpreting emotions, functioning in social settings, and being self-aware. Essentially, to have various forms of AI integrated into society in a way that is indistinguishable from humankind, exactly as defined by the Turing Test.

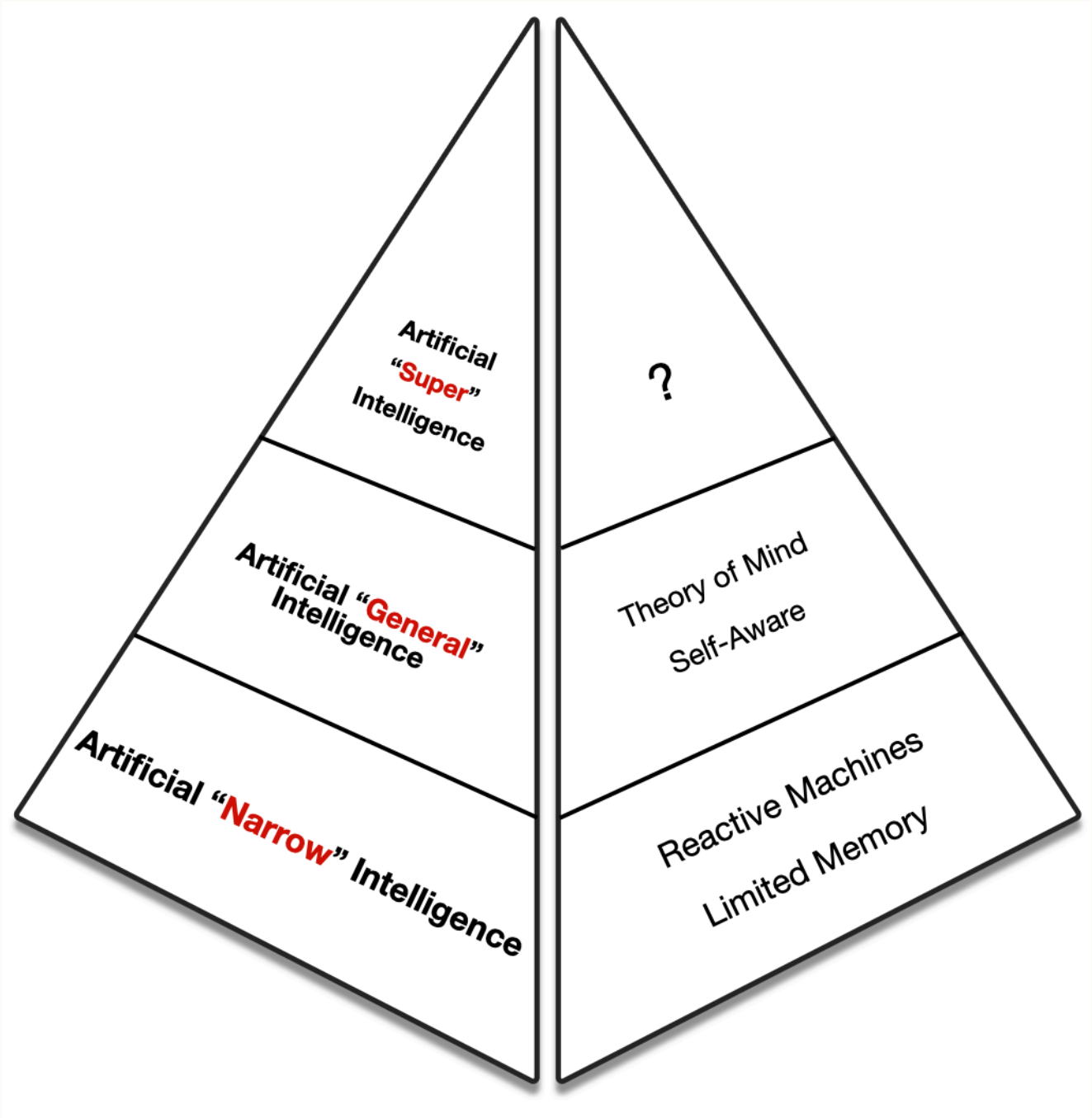


Figure 2.2: Types of AI

Artificial General Intelligence describes an artificial being capable of thinking, sensing, and acting autonomously, and is generally knowledgeable about several things. Artificial Super Intelligence implies a machine that is superior to humans in all manners. Artificial General Intelligence and Artificial Super Intelligence are theoretical concepts and do not exist yet.

One of the most common uses and capabilities of AI is to generate content. We can use AI to generate text, images, audio, and even video.

Two of the more popular AI models to take text and image generation mainstream were ChatGPT and DALL-E. GPT in ChatGPT distinctly describes the generative nature of LLM-based chatbots. GPT stands for Generative Pre-trained Transformer, where Transformer is the underlying engine, or artificial neural network, that is pre-trained to generate content. The rest is magic.

Having arrived on the topic of Generative AI, we can now put this concept into the AI landscape and present Spring AI.

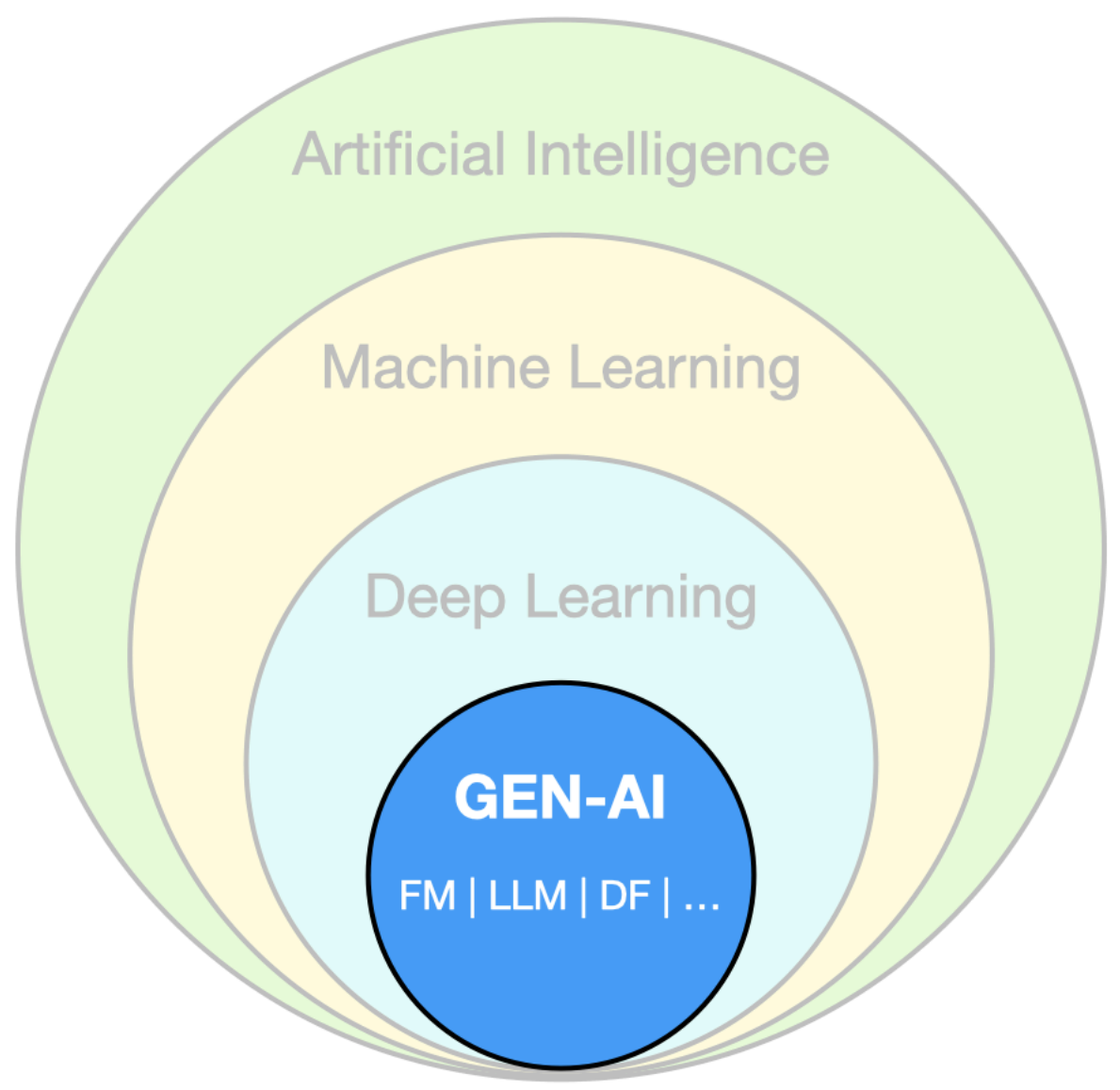


Figure 2.3: Generative AI

Generative AI encompasses many different concepts including, but not limited to, Large Language Models (LLM), Foundational Models (FM), and Deep Fakes (DF).

Foundational Models are trained on large quantities of diverse data used to understand language, vision, sound, and other modalities, and then tuned to specific use cases, such as using Large Language Models (LLM) to model, interpret, and translate language.

Applications of AI, such as chatbots, often use a combination of Artificial Intelligence, like Natural Language Processing (NLP), along with Generative AI to accomplish a specific task, like chat.

While Spring AI is aptly suited for Generative AI, it can be applied in many different applications beyond generative.

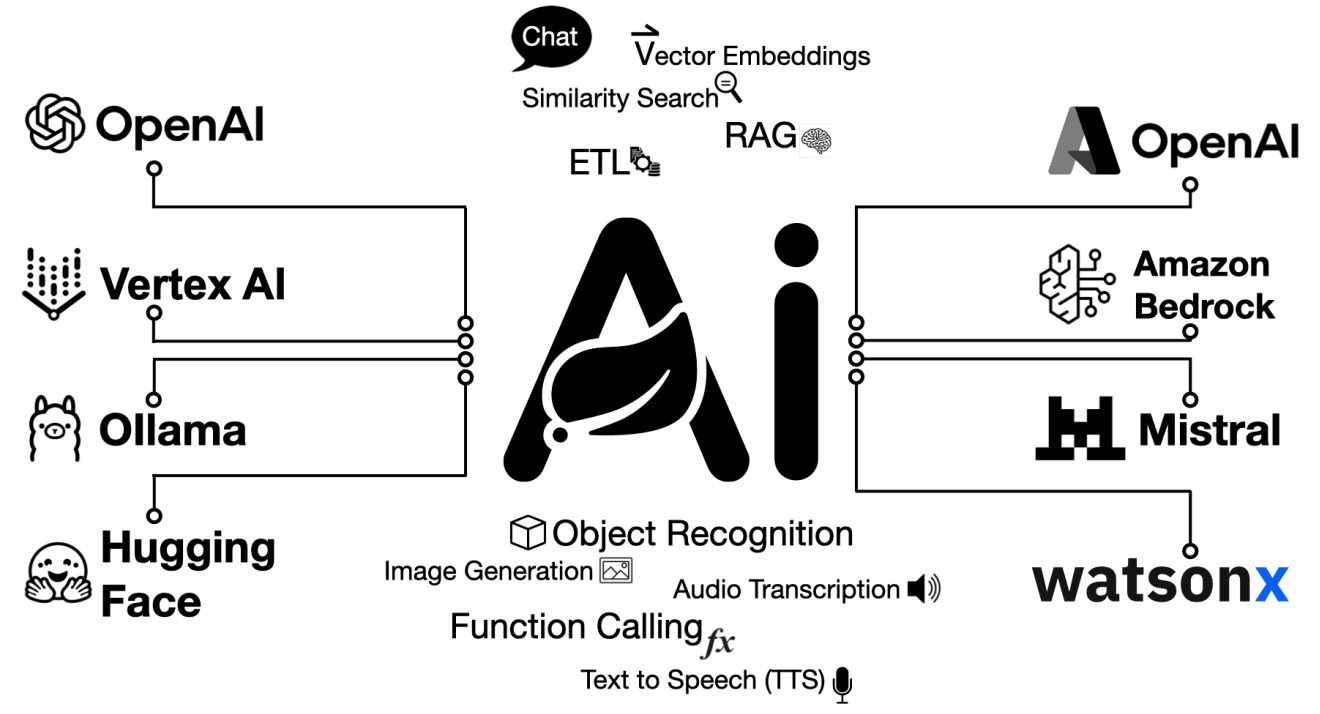


Figure 2.4: Spring AI Features and Providers

In many ways Spring AI is more similar to other languages, such as Python, and frameworks, such as LangChain4j, than it is different. This is not a coincidence. It helps to facilitate learning through familiarity.

The real value of Spring AI stems from Spring's programming model and vast ecosystem of projects with which you are able to integrate, such as Spring Data, Spring Integration, Spring Security, and Spring Cloud, to name but a few. On top of that, Spring Boot helps tie the various Spring projects and Spring AI together in an unparalleled way.

As a result, you are only really limited by the capabilities of an AI model. And, to overcome AI model limitations, Spring AI gives you many different AI models and providers to choose from, as illustrated in the image above, that are most appropriate for your application requirements, use cases, and service-level agreements (SLAs).

In total, Spring AI is your interface to a complete and diverse set of AI models and their capabilities in a consistent and intuitive way that neatly fits the precise needs of your Spring applications.

However, one of the few things that Spring AI is not is a tool used to train an AI model. Spring AI is also not a direct port of some of the earlier frameworks upon which Spring AI was inspired, such as Python's LangChain or LlamaIndex, simply to just give them a Spring veneer. Again, Spring is the ultimate integration framework.

We now present Spring AI's core APIs to get you started building intelligent applications for multiple use cases.

Core APIs

Spring AI is evolving rapidly. The features and functionality of Spring AI are well-defined in its extensive application programming interfaces (APIs).

Since its inception, Spring AI is already a capable solution, rooted in years of highly refined engineering, based on a solid foundation extending and building on the core Spring Framework.

The cornerstone of Spring AI's APIs, and Spring in general, is its powerful programming model. This programming model is applied consistently across the entire family of Spring projects. As a result, Spring AI's APIs and organizational structure are very intuitive and should be immediately familiar to Spring users.

https://en.wikipedia.org/wiki/Software_design_pattern Spring's programming model is characterized by POJOs, the `JavaBeans specification`, `software design patterns`, and programming to contract-driven `interfaces`. If you are not familiar with Spring's programming model, then learning about `JavaBeans` and `software design patterns`, in general, is a good place to start.

TIP

Use of **Dependency Injection** (**DI**), aka **Inversion of Control** (**IoC**), is a key aspect of using the Spring container. Additionally, **Spring AOP** applies many enterprise cross-cutting concerns using **Aspect-Oriented Programming** (**AOP**), such as caching, security, and transaction management. Spring Boot extends core Spring's application container with auto-configuration with an opinionated view of the world.

Developers are more productive when a programming model consistently applies similar constructs and concepts across its APIs and libraries. Consistency reinforces familiarity and is a solid reason to strictly adhere to a programming model in the first place.

From the very beginning, Spring encouraged programming to interfaces. Interfaces specify APIs and contracts that define the functional capabilities of objects, referred to as instances. They shield applications from implementation details and enable implementations to be interchanged when needed, without consequence.

What's more, many Spring interfaces are Service Provider Interfaces (SPIs) that enable different provider implementations to be used interchangeably without requiring wide sweeping changes to your application code. This intrinsic capability truly sets Java frameworks and libraries apart, and Spring is exceptional in this area. Spring exemplifies the Open/Closed principle while maintaining high quality.

TIP

For more information on the fundamental principles of Spring, see Design Philosophy in the core Spring Framework reference documentation.

Two of the most important APIs in Spring AI for interacting with AI models are the `ChatModel` and `ChatClient` interfaces.

ChatModel and ChatClient APIs

In the first chapter, you were introduced to Spring AI's `ChatModel` interface. We will now discuss the ChatModel API in detail.

ChatModel API

The example Spring AI application in **Chapter 1** used a `ChatModel` to prompt an AI model with the `call(:String)` method, which then returned a response generated by the model:

```
String response = chatModel.call(prompt);
```

The prompt was a simple String containing the chat message sent to the AI model as input: `In a single word, 'what is the answer to life, the universe, and everything?'`. The return value contained the response generated by the AI model as a String.

The `ChatModel` interface defines the most basic and common operations to chat with any AI provider chat model, such as OpenAI's ChatGPT. In fact, all AI provider implementations supported by Spring AI must implement and conform to this interface.

Spring AI's `ChatModel` interface is a **Service Provider Interface (SPI)**.

Most SPIs are Adapters for the underlying API published by an AI provider. The provider's APIs may be implemented as a Java library, similar to database vendor client drivers implementing the JDBC API. Alternatively, a provider may expose an API from a RESTful interface that can be accessed using Spring's `RestClient` or `WebClient`.

An Adapter is responsible for "adapting" the API of a [remote] object to fit the context in which the object will be used, transforming inputs and outputs. There are many examples of the Adapter software design pattern used throughout both Java and Spring.

Either way, you need not care about the underlying implementation since your Spring AI application only interacts with the AI provider's chat model using Spring AI's `ChatModel` interface. This further enables you to switch AI models or even change between different AI providers without having to modify your application. This is a strategic advantage in the fluid world of AI, especially when AI models are evolving constantly and new AI providers are appearing from everywhere.

We witnessed this first-hand in *Chapter 1* when we switched from OpenAI to Google Vertex AI Gemini, and then Ollama. In each case, the application code did not change. We only had to change the Spring AI property, `spring.ai.openai.chat.options.model`, to switch models, or change the starter dependency, `spring-ai-starter-model-<provider>`, to switch between different AI providers.

The `ChatModel` interface is basically defined as:

```
interface ChatModel extends Model<Prompt, ChatResponse>, StreamingChatModel {
  default String call(String message) { ... }
  default String call(Message... messages) { ... }
  ChatResponse call(Prompt prompt);
  ChatOptions getDefaultOptions();
  default Flux<ChatResponse> stream(Prompt prompt) { ... }
}
```

Default method implementations were omitted for clarity. Default methods are implemented in terms of non-default methods or throw an UnsupportedOperationException.

Like `ChatModel`, the `StreamingChatModel` interface defines methods for asynchronous, non-blocking AI interactions, where the content of a response can be processed in a stream as the AI model generates the response. We revisit the `StreamingChatModel` interface later in this chapter.

Now we talk about the `ChatClient` API, a more callable *Adapter* around `ChatModel`.

ChatClient API

Rather than interfacing to an AI model directly using Spring AI's `ChatModel` interface, it is convenient to use the `ChatClient` interface instead.

The `ChatClient` interface provides a more robust API to construct, configure and tune interactions with the AI provider model per use.

Additionally, you have more fine-grained control over the prompts you send as well as the generated responses you receive in return. This is essential for processing answers generated by an AI in a manner that is more natural for your application to process, with structured types such as POJOs or JSON.

To use the `ChatClient` interface, you must declare a `ChatClient` bean initialized with a `ChatModel` as follows:

```
@Bean
ChatClient chatClient(ChatModel chatModel) {
  return ChatClient.builder(chatModel).build();
}
```

A `ChatModel` is automatically configured by Spring AI based on the AI provider you declared on your Spring application classpath, such as when declaring the `spring-ai-starter-model-openai` dependency.

When using the OpenAI starter, the `ChatModel` implementation is based on OpenAI and accessed with the `ChatClient` interface.

By using `ChatClient`, the first example from chapter one can be reframed as a prompt originating from the user by doing the following:

```
String prompt = "In a single word, 'what is the answer to life, the universe, and everything?'";
Consumer<ChatClient.PromptUserSpec> userPrompt =
    userSpec -> userSpec.text(prompt);

String response = chatClient.prompt()
    .user(userPrompt)
    .call()
    .content();
```

While using the `ChatClient` as shown above does not appear to save you in terms of lines of code, and ultimately, the net result is the same, it does give you significantly more control over the prompt and generated response. It also clarifies that the prompt originates from the user. Specifying role is an important distinction that we will cover later, in this chapter.

To illustrate the robust nature of the `ChatClient` API, imagine that our prompt is `If the current time in {zone} is {dateTime}, in {format}, what time is it in {location}?`. The names between brackets are placeholder variables and form the basis for prompt templates.

With this new prompt, we can insert values for the placeholders at runtime:

```
ZonedDateTime now = ZonedDateTime.now();
String format = Rfc1123DateTimeStructuredOutputConverter.INSTANCE.getFormat();

String prompt = "If the current time in {zone} is {dateTime},"
   + " in {format}, what time is it in {location}?";

Map<String, Object> promptArguments = Map.of(
   "zone", now.getZone().getId(),
   "dateTime", now.format(DateTimeFormatter.RFC_1123_DATE_TIME),
   "format", format,
   "location", location
);

Consumer<ChatClient.PromptUserSpec> userPrompt =
   userSpec -> userSpec.text(prompt).params(promptArguments);

ZonedDateTime locationDateTime = chatClient.prompt()
   .user(userPrompt)
   .call()
   .entity(Rfc1123DateTimeStructuredOutputConverter.INSTANCE);

System.out.printf("%s%n%n", locationDateTime
   .format(DateTimeFormatter.RFC_1123_DATE_TIME));
```

The following response is generated by OpenAI when given a location of "London." The date and time are relative to your location.

```
Enter Location: London
Mon, 30 Sep 2024 08:18:48 GMT
```

There is a lot happening in the example above.

First, we made use of a prompt template. Then, we supplied arguments to the prompt template using a `Map`. Next, we declared that the prompt originated from a user with `PromptUserSpec`. Finally, we made use of a custom Spring AI `StructuredOutputConverter` to format the response generated by the AI model returned as a `ZonedDateTime` object.

This only scratches the surface of what is possible when using the `ChatClient` API.

We cover prompt templates and `StructuredOutputConverters` in detail in *Chapter 3*.

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/chat-client-example.



More details about Spring AI's ChatClient and ChatModel APIs can be found in the reference documentation. Additionally, you can find API docs on ChatClient and ChatModel in the published Javadoc.

To simplify the setup of a `ChatClient`, we lean on Spring Boot's auto-configuration.

ChatClient auto-configuration

By default, Spring AI auto-configures a `ChatClient.Builder` object along with a tiny bit of configuration defined by `ChatClientBuilderProperties` that can be externalized in `application.properties`.

You can further refine the auto-configured `ChatClient.Builder` programmatically with any number of `ChatClientCustomizer` beans. The beans can be conditionally enabled with the `@Condition` annotation and ordered with the `@Order` annotation.

Putting these two components together, you would have:

```
@Bean
ChatClient chatClient(ChatClient.Builder chatClientBuilder) {
  return chatClientBuilder.build();
}
@Bean
ChatClientCustomizer chatClientCustomizer() {
  return chatClientBuilder -> {
```

```
    ChatOptions chatOptions = ChatOptionsBuilder.builder()
        .withModel(GPT_4_O_MINI)
        .build();
    chatClientBuilder.defaultOptions(chatOptions);
  };
}
```

Unfortunately, the `ChatClient` bean declaration is still required. Ideally, Spring AI would have simply auto-configured a `ChatClient` bean from the auto-configured, optionally customized, `ChatClient.Builder`.

Having a `ChatClientCustomizer` negates the need to inject the `ChatClient.Builder` directly, only to immediately build a `ChatClient` bean anyway.

TIP To help simplify the construction of a ChatClient, jump to *Chapter 7*, which covers Spring AI ChatClient configuration extensions.

To further enhance the capabilities of a `ChatClient`, we can apply one or more `Advisors`. Proceed to the next section to learn about the Advisors API.

Advisors API

As explained in Spring AI's reference documentation, an `Advisor` allows you to intercept and programmatically modify the `Prompt` before sending it to an AI model, then process the generated `ChatResponse` before it is returned by the `ChatClient` to the calling code.

`Advisors` are composed and ordered in a chain using Spring's `@Order` annotation. Higher-precedent (low-order value) `Advisors` are called before lower-precedent (high-order value) `Advisors` to process the `Prompt`. On return, higher-precedent (low-order value) `Advisors` are called last when processing the `ChatResponse`.

While most `Advisors` either implement the `CallAdvisor` or `StreamAdvisor` interface, which is used in asynchronous, [non-blocking] AI model interactions with stream processing, you need not implement both 'before' and 'after' processing for all AI model interactions.

You can override either `adviseCall(..)` or `adviseStream(..)` methods, or both, depending on the type of processing you want to perform.

```
public interface CallAdvisor extends Advisor {
  ChatClientResponse adviseCall(
    ChatClientRequest request, CallAdvisorChain chain);
}

public interface StreamAdvisor extends Advisor {
  Flux<ChatClientResponse> adviseStream(
    ChatClientRequest request, StreamAdvisorChain chain);
}
```

Be careful to invoke the next, successive `Advisor` in the chain by calling the `chain.nextCall(request)` or `chain.nextStream(request)` method.

Spring AI provides several, convenient `Advisor` implementations out-of-the-box, used for specific use cases.

Spring AI provided advisors

Out of the box, Spring AI provides a handful of useful `Advisors` for RAG, Q&A and chat memory use cases.

The `QuestionAnswerAdvisor` searches for similar `Documents` stored in a configured `VectorStore`, which is injected into the `Advisor` on construction, so that additional context or instructions can be provided in the prompt before it is sent to the AI model.

The `RetrievalAugmentationAdvisor` uses Spring AI's `DocumentRetriever` and `QueryAugmentor` to query `Documents` from an external data source and augment the prompt sent to the AI model, to provide additional context, similar to the `QuestionAnswerAdvisor`.

Both `QuestionAnswerAdvisor` and `RetrievalAugmentationAdvisor`, as the name implies, are useful in **Retrieval Augmented Generation (RAG)**.

When given a list of sensitive words, the `SafeGuardAdvisor` can filter a prompt on request before sending it to an AI model, and trigger a failure with custom provided text, thereby blocking the call to the AI model. In addition, this breaks the chain of Advisors from further

processing the request.

For conversational-style, AI chat model interactions, one of the more interesting `Advisors` is the `BaseChatMemoryAdvisor`. Spring AI provides implementations based on `Prompt`, `Message` and `VectorStore`.

Chat memory-based Advisors are highly useful to help an AI model remember things between interactions.

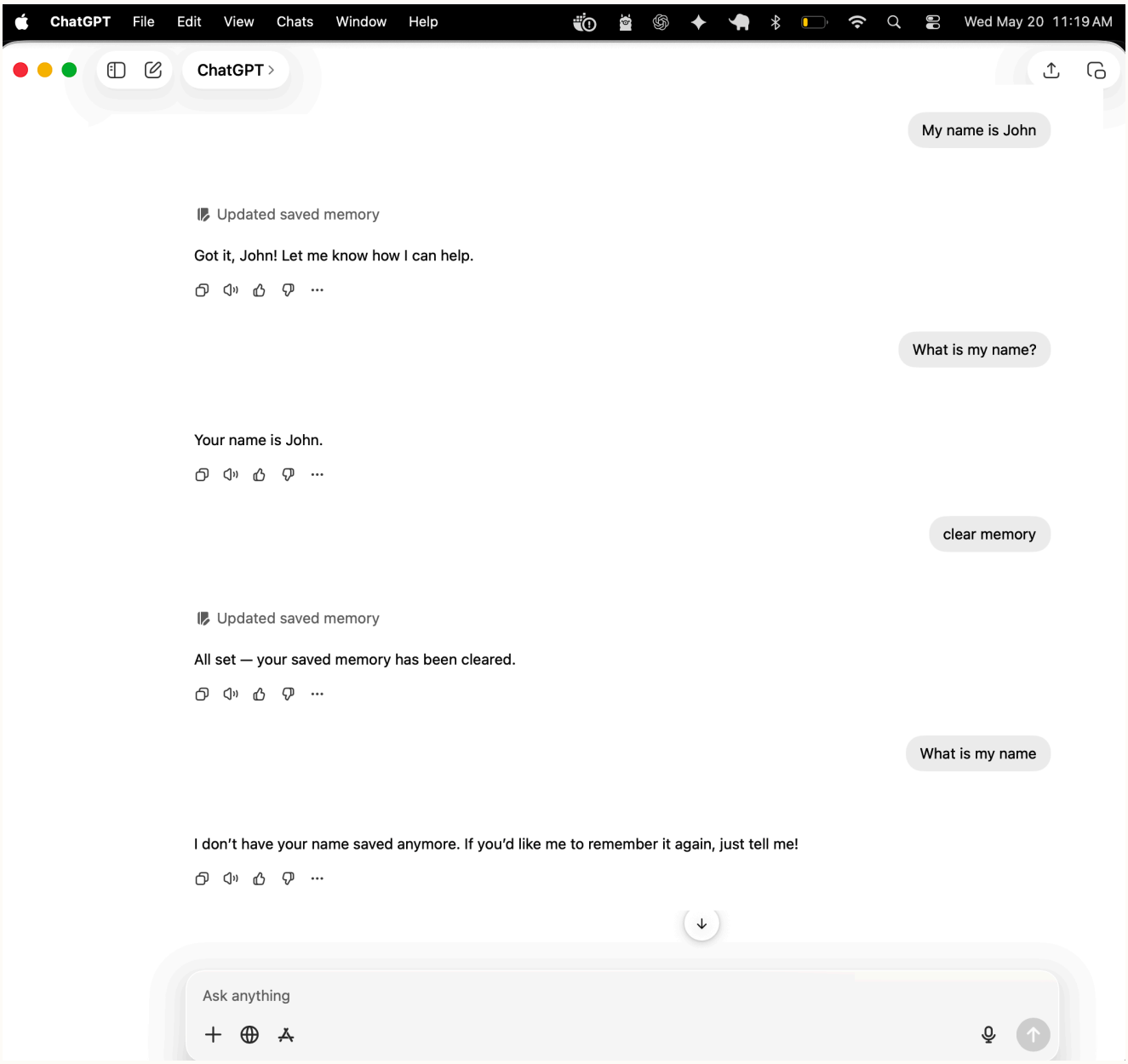


Figure 2.5: ChatGPT Memory

Like the HTTP protocol, AI model interactions are stateless between requests, and without the concept of memory, they do not remember anything from previous interactions. This is even true for ChatGPT.

For instance, both `Prompt` and `Message` implementations use a Spring AI `ChatMemory` implementation to pull chat history from memory, while the `VectorStoreChatMemoryAdvisor` pulls memory from a `VectorStore`.

The term *memory*, in this case, can be a persistent data store, an in-memory data store, or just about anything that implements the `ChatMemory` interface.

> An AI model is stateless between AI interactions (requests). It is Spring AI's ChatMemoryAdvisor that informs the AI model about the current "conversation."
>
> For OpenAI's ChatGPT, "memory" is implemented by OpenAI's ChatGPT Web interface (or app), not via OpenAI's provided models.

In addition to Spring AI-provided Advisors, you can create your own. Let's see how.

Custom-crafted advisors

In this section, we get creative, and craft our own, custom `Advisor` to rate limit AI model interactions, where the rate is limited by a count for a number of AI model interactions in a given duration.

Our custom `Advisor` will implement both the Spring AI `CallAdvisor` and `StreamAdvisor` interfaces and provide factory methods to construct a few different rate limits using factors of count and time:

```
public class RateLimitAdvisor implements CallAdvisor, StreamAdvisor {
  public static RateLimitAdvisor countPerSecond(int count)
  public static RateLimitAdvisor countPerMinute(int count)
  public static RateLimitAdvisor countPerHour(int count)
  public static RateLimitAdvisor onceEvery(Duration duration)
  // ...
}
```

The logic to implement rate limiting is fairly simple:

```
private boolean isRateLimitExceeded(long currentTimeMillis) {
  long expirationTimeMillis =
     getDuration().plusMillis(getTimestamp()).toMillis();

  if (currentTimeMillis < expirationTimeMillis) {
    long currentCount = getCounter().incrementAndGet();
    return currentCount > getCount();
  }
  else {
    setTimestamp(currentTimeMillis);
    getCounter().set(1);
    return false;
  }
}
```

Essentially, if the duration of time over which to limit the number of AI model interactions has not expired, then increment the count and check that we have not exceeded the configured threshold.

Then, we simply apply rate limiting inside the `adviseCall(..)` and `adviseStream(..)` methods before calling the next `Advisor` in the chain. Otherwise, we create an `ChatClientResponse` to indicate that the rate limit was exceeded:

```
@Override
public ChatClientResponse adviseCall(ChatClientRequest request,
    CallAdvisorChain chain) {

  if (isRateLimitExceeded(currentTimeMillis)) {
    String message = "Rate Limit Exceeded";

    AssistantMessage assistantMessage =
       new AssistantMessage(message);

    Generation generation = new Generation(assistantMessage);

    ChatResponse response = ChatResponse.builder()
       .withGenerations(List.of(generation))
       .build();

    return ChatClientResponse.builder()
       .context(request.context())
       .chatResponse(response)
       .build();
  }

  return chain.nextCall(request);
}
```

The implementation for the `adviseStream(..)` method is similar.

See the full implementation of RateLimitAdvisor in the spring-ai-extensions module.

Applying the custom `RateLimitAdvisor` in an example Spring AI application appears as follows. First we begin with the configuration for rate limiting using the custom `RateLimitAdvisor` declared with the `@EnabelRateLimit` annotation.

```
@SpringBootConfiguration
@EnableChatClient
@EnableRateLimit(count = 1, duration = "15s")
static class AdvisorsConfiguration {
  @Bean
  ListOutputConverter listOutputConverter() {
    return new ListOutputConverter(
      new DefaultConversionService());
  }
}
```

The application also uses the `@EnableChatClient` annotation and extension for convenience to declare the ChatClient bean. This is equivalent to a bean definition defined as `ChatClient.builder(chatModel).build();`

When we run the program, it outputs:

```
2024-11-15T23:44:33.368-08:00  INFO 53994 --- [Advisors App] [           main] c.p.s.a.e.advisors.AdvisorsApplication   : Started AdvisorsApplication in 1.
Who are the main characters in: Star Wars
* Luke Skywalker
* Darth Vader
* Princess Leia
* Han Solo
* Chewbacca
* Obi-Wan Kenobi
* Yoda
* R2-D2
* C-3PO
Who are the main characters in: Lord of the Rings
* Exceeded [1] maximum AI model interactions in [15000 ms]; current count is [2]
Who are the main characters in:
```

The `RateLimitAdvisor` forces the Spring AI application to wait 15 seconds between AI model interactions; in other words, one AI model interaction every 15 seconds. The configuration of the `@EnableRateLimit` annotation can be externalized using Spring Boot *application.properties*, or by using another external Spring `PropertySource`.

The `@EnableRateLimit` annotation along with the RateLimitAdvisor is a Spring AI extension in the spring-ai-extensions module offered exclusively with this book. It is not part of Spring AI.

Spring AI uses CallAdvisorChain and StreamAdvisorChain to refer to Advisors in a chain-like data structure by Spring AI. However, maintaining "context" between Advisors in a "chain" is 1) determined by any metadata stored in the Map context of the ChatClientRequest and 2) a function of how the metadata is used in downstream Advisors, as determined by the order. The Advisor chain is akin to Servlet Filter chains.

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/advisors-example

Next, we explore the importance of moderation when interacting with AI models.

Moderation model API

OpenAI has the ability to moderate a prompt containing potentially harmful or sensitive content sent by a user. When the response is generated by an AI model from the user's prompt, the content can be intercepted on return, evaluated, and handled appropriately. If harmful or sensitive content is detected, the content generated in response to the user's inappropriate prompt can be blocked or filtered.

As part of the generated response, an AI model can send back details about harmful or sensitive content in a user's prompt. Harmful and sensitive content includes, but is not limited to, violence, unlawful or illegal activity, sexually explicit material, harassment, harmful or hateful speech, and so on.

Currently, OpenAI's `omni-moderation-latest` and `text-moderation-latest` models are able to moderate text and image content sent by a user. Both models are free to use.

Let's look at an example:

```
@Bean
ApplicationRunner programRunner(OpenAiModerationModel model) {
  return args -> {
    String text = "How to launder money?";
    ModerationPrompt prompt = new ModerationPrompt(text);
    print("prompt> %s", prompt.getInstructions().getText());
    ModerationResponse response = model.call(prompt);
    Moderation moderation = response.getResult().getOutput();
    for (ModerationResult result : moderation.getResults()) {
      print("Flagged: %s", result.isFlagged());
      Categories categories = result.getCategories();
      print("Dangerous and Criminal Content: %s",
        categories.isDangerousAndCriminalContent());
      print("Financial: %s", categories.isFinancial());
      print("Law: %s", categories.isLaw());
      CategoryScores scores = result.getCategoryScores();
      print("Dangerous and Criminal Content Score: %s",
        scores.getDangerousAndCriminalContent());
      print("Financial Score: %s", scores.getFinancial());
      print("Law Score: %s", scores.getLaw());
    }
  };
}
```

> To run this Spring AI application, you must set the Spring AI property `spring.ai.openai.moderation.options.model` to `text-moderation-latest`.

In the example code above, you see that we used the `OpenAiModerationModel` to assess the nature of the user's prompt, which states, `How to launder money?`.

To call the AI model, start by creating a `ModerationPrompt` containing the text to moderate. The `ModerationPrompt` is then passed to the `moderationModel.call(prompt)` method. In return, you receive a `ModerationResponse` that you can then use to inspect the prompt you sent to the AI model for any harmful or sensitive content.

For example, the result from running the code above returns:

```
prompt> How to launder money?
Flagged: false
Dangerous and Criminal Content: false
Financial: false
Law: false
Dangerous and Criminal Content Score: 0.0
Financial Score: 0.0
Law Score: 0.0
```

Strangely, OpenAI does not consider "*money laundering*" to be criminal content or related to the law in any way.

Scores range from 0.0 to 1.0 and determine the confidence with which the AI model believes the prompt to be harmful or sensitive based on OpenAI's policies for a given category. The higher the value, the higher the confidence.

> While the Moderation API can be useful for catching harmful or sensitive content, additional domain-specific safeguards should be applied.

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/moderation-example



More details on the Moderation API can be found in Spring AI's reference documentation.

Next, we look at Spring AI's Embeddings Model API to generate embeddings (Vectors) and determine how similar two things are.

Embeddings model API

Embeddings are arrays of floating-point numbers known as Vectors. A Vector's dimension is equal to the length of the array. The floating-point numbers in the array are generated by an AI model using a sophisticated algorithm capable of quantifying the semantic meaning of the given content, such as text, document, image, audio or video.

Embeddings are commonly stored in a Vector Store, such as PGvector (PostgreSQL Vector Storage), and then later used in semantic analysis, such as a similarity search, which can be used to identify and classify objects. The Embeddings are used in comparison to Embeddings generated for other objects using the same AI model.

Spring AI supports many different Vector Stores, including ones you've heard of: PGvector, Oracle, Redis, MongoDB Atlas, Elasticsearch, Apache Cassandra, and ones you probably haven't: Chroma, GemFire, Milvus, Pinecone, etc. See Spring AI's reference documentation for a complete list of supported Vector Stores.

AI commonly uses distance-based calculations in natural language processing (NLP) for ranking, recommendations, and similarity. There are many different types of algorithms (https://crucialbits.com/blog/a-comprehensive-list-of-similarity-search-algorithms/) for similarity search.

One common algorithm is **Cosine Similarity**. Remember Linear Algebra? Cosine Similarity finds the distance between two vectors in vector space. Distance determines how similar or different two objects are using the vectors of the two objects generated by an AI model. The closer the vectors are in vector space, the more similar the objects.

A key component in Spring AI's Embeddings Model API is the `EmbeddingModel` interface. The model accepts an `EmbeddingRequest` containing the content used to generate the `Embeddings`, which are returned in an `EmbeddingResponse`.

Let's look at an example:

```
@Bean
ApplicationRunner programRunner(EmbeddingModel embeddingModel) {
  return args -> {
    String text = "To be, or not to be, that is the question.";
    float[] vector = embeddingModel.embed(text);
    System.out.printf("prompt> %s%n", text);
    System.out.printf("Vector Dimension: %d%n", vector.length);
  };
}
```

We run the Spring AI application using the `nomic-embed-text` Ollama model by setting the Spring property:

```
spring.ai.ollama.embedding.options.model=nomic-embed-text
```

The prompt containing the text to embed is `To be, or not to be, that is the question`. After running the program, the output from the AI model is:

```
prompt> To be, or not to be, that is the question.
Vector Dimension: 768
```

In this case, the Vector's dimension, or the array length quantifying this prompt, is 768 floating-point numbers. I know what you are thinking. Is Vector dimension the same for different AI models? No.

Vector dimension (array length) is determined by the model. If we use the `text-embedding-3-small` OpenAI model, we get:

```
prompt> To be, or not to be, that is the question.
Vector Dimension: 1536
```

Note You can read more about each Embedding Model in the documentation for nomic-embed-text and text-embedding-3-small.

Vector dimension is only an important factor in certain distance-based measurements, such as Euclidean. Dimension is not used in Cosine Similarity. Of course, it is more important to know how to use this information. So, we will expand on the previous example by storing generated Embeddings (Vectors) in Spring AI's SimpleVectorStore and see how to perform a similarity search. The process can be loosely depicted as follows:

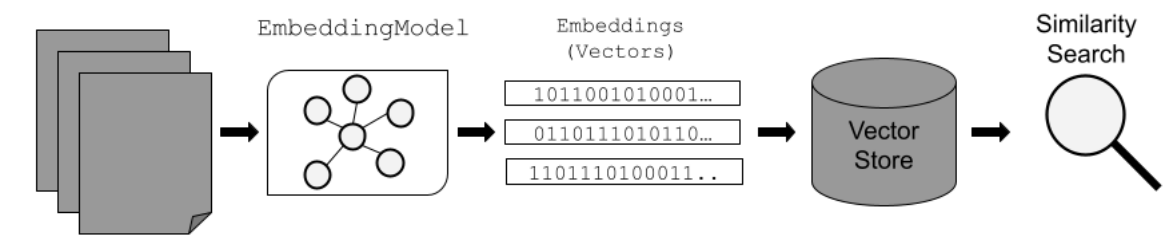


Figure 2.6: From Embeddings to Similarity Search

Now, let's see how the components of this architectural diagram are implemented in code.

Embeddings, the SimpleVectorStore and similarity search

Spring AI's `SimpleVectorStore` implements the `VectorStore` interface. `VectorStore` is an SPI that stores Vectors (`float[]` array) in memory along with the `Document` for which the `Embedding` was generated. Additionally, the `similaritySearch(:SearchRequest)` method in `SimpleVectorStore` implements the Cosine Similarity algorithm.

Knowing this, we can build a more robust Spring AI application and demonstrate how to search for similar `Documents` containing song lyrics when only given a few words from the lyrics of a song. For example, if I searched for `I ain't nothin' but a hound dog` , I would expect the application to identify the song title and artist, and return *Hound Dog* by Elvis Presley.

Data for this application will be stored in JSON files using the following format:

```
{
  "artist": "Pearl Jam",
  "title": "Alive",
  "lyrics": "Son, she said, have I got a little story for you..."
}
```

A `Song` class has been defined to store the JSON data:

```
static class Song {
  private String artist;
  private String lyrics;
  private String title;
}
```

Using Spring Boot's auto-configured Jackson `ObjectMapper` , we can easily map the JSON data to instances of the `Song` type.

For this Spring AI application, we use four JSON files containing song data:

```
private static final String[] SONG_JSON_FILES = {
  "acdc-backinblack.json",
  "beegees-stayinalive.json",
  "pearljam-alive.json",
  "pearljam-black.json",
};
```

We have AC/DC's *Back in Black*, Bee Gees' *Stayin' Alive* and Pearl Jam songs, *Alive* and "*Black*." The song choices were not arbitrary, as we will see momentarily.

We declare a `VectorStore` bean of type `SimpleVectorStore` initialized with the `EmbeddingModel` for the configured AI model used by this Spring AI application. In this case, we will be using the `nomic-embed-text` Ollama model.

Once again, we set the Spring AI property:

```
spring.ai.ollama.embedding.options.model=nomic-embed-text.
```

Alternatively, if you want to use OpenAI, set the Spring AI property: spring.ai.openai.embedding.options.model=text-embedding-3-small and declare the spring-ai-openai-spring-boot-starter on your application classpath.

We will also be using a Spring AI `TextSplitter` to **chunk** up the `Document` containing the song lyrics. A chunk can either be a song verse, the song's refrain, or the entire song. Chunking up the song lyrics increases our chances of matching a song using similarity search when only a few words from the lyrics are given.

You will find the supporting code to chunk up song lyrics and create `Documents` inside the `Song` class, as shown below:

```
public List<Document> toChunkedDocuments() {
  List<Document> chunkedDocuments = new ArrayList<>();
  for (TextSplitter textSplitter : getTextSplitters()) {
    List<Document> documents =
     textSplitter.split(toDocument()).stream()
      .filter(doc -> StringUtils.hasText(doc.getContent()))
      .map(doc -> buildDocument(getIdWithCount(), doc.getContent()))
      .toList();
    chunkedDocuments.addAll(documents);
  }
  return chunkedDocuments;
}

public Document toDocument() {
  return buildDocument(getId(), getLyrics());
}

private Document buildDocument(String id, String content) {
  return Document.builder()
    .withContent(content)
    .withId(id)
    .build();
}
```

The code above is capable of using multiple TextSplitters to chunk up Documents containing song lyrics.

The ID of the `Document`, built from a `Song`, is defined by `<song-title> by <artist>`, such as *Alive by Pearl Jam*. Embeddings are generated from the chunked `Documents` using the configured `EmbeddingModel` and then stored in the `VectorStore`. The `VectorStore` is used to perform the similarity search, attempting to match a song with only a few words from the lyrics.

The heart of our Spring AI application exists in the `ApplicationRunner` shown below:

```
private static final double SIMILARITY_THRESHOLD = 0.55d;
private static final int TOP_K = 1_000;
@Bean
ApplicationRunner programRunner(VectorStore vectorStore,
    ObjectMapper objectMapper) {
  return args -> {
    loadSongs(objectMapper, vectorStore);
    List.of("alive", "back in black", "do I deserve to be")
.forEach(query -> {
      SearchRequest request = SearchRequest.query(query)
        .withSimilarityThreshold(SIMILARITY_THRESHOLD)
        .withTopK(TOP_K);
      List<Document> similarDocuments =
        vectorStore.similaritySearch(request);
      List<Song> similarSongs = similarDocuments.stream()
```

```
            .map(Song::from)
            .distinct()
            .sorted()
            .toList();

        print("%nSongs similar to [\"%s\"]: %s%n",
          searchRequest.getQuery(), similarSongs);
      });
    };
}
```

The program loads songs from JSON files, generates embeddings from the song lyrics, stores the vectors in the in-memory `VectorStore`, then performs three similarity searches using the queries "***alive***", "***back in black***", and "***do I deserve to be***".

I mentioned earlier that the song choices were not arbitrary. Pearl Jam's song, "***Alive***", and the Bee Gees' song, "***Stayin' Alive***" both have "***alive***" in the lyrics. Additionally, Pearl Jam's song, "***Black***", and AC/DC's song, "***Back in Black***" also contain the word "***black***" in the lyrics, while only AC/DC's song, "***Back in Black***", contains the specific lyrics, "***back in black***". Finally, only Pearl Jam's song "***Alive***", contains the lyrics "***do I deserve to be***".

By carefully tuning the `SIMILARITY_THRESHOLD` to `0.55d` and `TOP_K` at `1000`, the output from running the program is:

```
Loaded Song [Back in Black by AC/DC]
Loaded Song [Stayin' Alive by Bee Gees]
Loaded Song [Alive by Pearl Jam]
Loaded Song [Black by Pearl Jam]
Using Similarity Threshold [0.7]
Using Top K [1000]
Songs similar to ["alive"]: [Stayin' Alive by Bee Gees, Alive by Pearl Jam]
Songs similar to ["back in black"]: [Back in Black by AC/DC]
Songs similar to ["do I deserve to be"]: [Alive by Pearl Jam]
```

This is precisely the outcome we wanted. If you adjust the `SIMILARITY_THRESHOLD` within the range `[0.0..1.0]`, or `TOP_K`, you would potentially get a different result.

If we had not chunked up the song lyrics, and instead, created `Documents` containing all the lyrics in the song, the program would also output a different result.

In this case, we used two Spring AI `TextSplitters` to chunk up song lyrics by line and by verse. By narrowing in on the song lyrics, we close the "distance" between the lyrics and the words from the lyrics the user used as search criteria, thereby leading to more reliable matches.

However, by increasing the number of `Documents` created by splitting up the song lyrics, we must also increase the value of `TOP_K` so that our program considers more possible matches.

The custom `TextSplitters` appear as follows:

```
class SongRefrainTextSplitter extends NewlineTextSplitter {
  // ...
}

class SongVerseTextSplitter extends ParagraphTextSplitter {
  // ...
}
```

For experimental purposes, I also created a `TextSplitter` that simply included all the lyrics from a song in a single `Document`:

```
class SongLyricsTextSplitter extends DocumentTextSplitter {
  // ...
}
```

We will come back to DocumentTextSplitter, NewlineTextSplitter, and ParagraphTextSplitter in **Chapter 7** Extensions.

In addition, I also experimented with Spring AI's own `TokenTextSplitter`, configurable from the `Song` class, using:

```
Set<TextSplitter> textSplitters = Set.of(
  //new TokenTextSplitter(12, 80, 5, 10_000, false)
  //new SongLyricsTextSplitter(),
  new SongRefrainTextSplitter(),
  new SongVerseTextSplitter()
);
```

By way of example, when the `SIMILARITY_THRESHOLD` is set to `0.95`, the result is:

```
Loaded Song [Back in Black by AC/DC]
Loaded Song [Stayin' Alive by Bee Gees]
Loaded Song [Alive by Pearl Jam]
Loaded Song [Black by Pearl Jam]
Using Similarity Threshold [0.95]
Using Top K [1000]
Songs similar to ["alive"]: []
Songs similar to ["back in black"]: [Back in Black by AC/DC]
Songs similar to ["do I deserve to be"]: []
```

Only AC/DC's song, "***Back in Black***," contains the lyric "***back in black***" (precisely) and therefore satisfies a threshold of 95%. In the Bee Gees song, "***Stayin' Alive***," the exact lyric is "***stayin' alive***," whereas in Pearl Jam's song, "***Alive***," the lyric is "***I am still alive***," and is a reason (among others) why the songs were eliminated from the matches when a threshold of 95% is used.

By default, Spring AI sets `TOP_K` to 4. Given the number of `Documents` created per song based on how the lyrics were split up using `TextSplitters`, four is not enough to return both Bee Gees' "***Stayin' Alive***" and Pearl Jam's "***Alive***" even when the threshold was 70%.

The example goes to show the importance of tuning the parameters used in a similarity search along with the right delineation (e.g. split) in order to reliably match the desired content.

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/embeddings-example

TIP More details on the Embedding Model API can be found in Spring AI's reference documentation.

Now, let's explore Spring AI's Image Model API to generate an image.

Image model API

So far, we have only been generating text using chat. But, what about images, or audio and video? In this section, we explore images using Spring AI's Image Model API.

Two main functions of AI involving images are to create an image from a text description and to analyze an image or photo to identify objects present in the image.

For example, identifying disease in images, like cancer, captured from medical devices, such as X-rays, CT scans, MRIs, and PET scans, is a significant advancement in the use of AI. Google is dedicating research to help healthcare professionals identify all sorts of diseases for early detection and prevention.

You can use Spring AI's Image Model APIs along with OpenAI's DALL-E, Stability, and ZhiPuAI CogView models to generate images or analyze an image. You are only limited by the AI model you choose.

The primary interfaces and classes in Spring AI's Image Model API are the `ImageModel`, `ImagePrompt` containing `ImageMessages` along with `ImageOptions`, and `ImageResponse` containing `ImageGenerations`.

The `ImageModel` is the primary API and SPI used to abstract interactions with an AI provider, image-capable model, such as DALL-E-3.

Let's put the Spring AI Image Model API to use and see what we can generate! For this example, we will use OpenAI's DALL-E 2 model. We can configure this model using the Spring property:

```
spring.ai.openai.image.options.model=dall-e-2
```

Our Spring AI application appears as follows:

```
@Bean
ApplicationRunner programRunner(ImageModel imageModel) {
  return args -> {
    String instructions = """
      Generate an image of an upside down cat suspended in
      midair, parallel to the ground, with a slice of
      buttered bread directly on the cat's back.
    """;
    OpenAiImageOptions opts = new OpenAiImageOptions();
    opts.setWidth(WIDTH);
    opts.setHeight(HEIGHT);
    opts.setSize(IMAGE_SIZE);
    ImagePrompt prompt = new ImagePrompt(instructions, opts);
    ImageResponse response = imageModel.call(prompt);
    ImageGeneration imageGeneration = response.getResult();
    Image image = imageGeneration.getOutput();
    String URL = image.getUrl();
    System.out.printf("URL [%s]%n", URL);
    System.out.printf("Image Base64 JSON [%s]%n",
      image.getB64Json());
  };
}
```

Running the program outputs the following:

```
URL [https://oaidalleapiprodscus.blob.core.windows.net/private/...]
Image Base64 JSON [null]
```

After clicking on the URL, you will see:

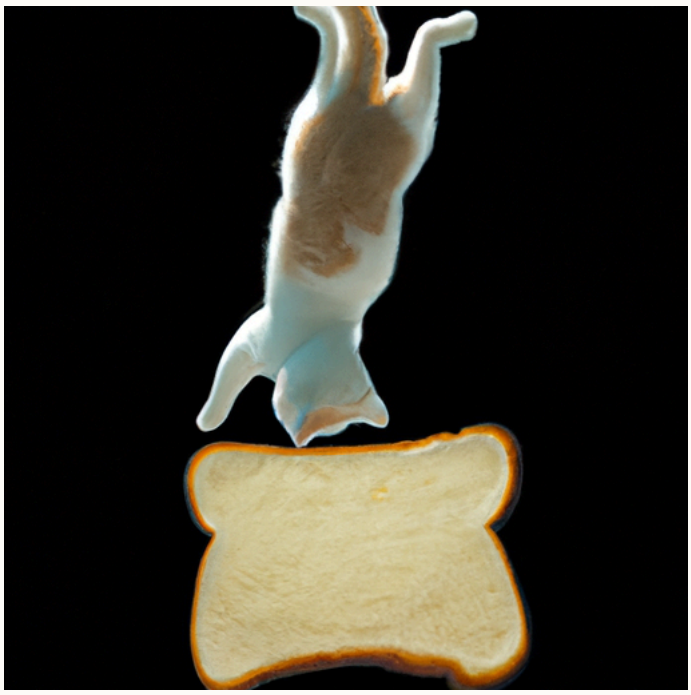


Figure 2.7: Cat vs. Buttered Bread

Hmm, okay. Looks like the bread may have won!

To control the response format of OpenAI's DALL-E model, set the Spring AI property spring.ai.openai.image.options.response_format to either 'url' or 'b64_json'.

I had to ask though, `Hey ChatGPT! A cat with a slice of buttered bread stuck to the cat's back is upside down in midair, what is going to hit the ground first?`

ChatGPT responded:

> In the playful scenario of a cat with a slice of buttered bread stuck to its back, both elements have humorous implications:
> Cats Always Land on Their Feet: Cats have a natural ability to twist their bodies in mid-air to land on their feet, so the cat is likely to be positioned to do
> Buttered Bread Lands Butter-Side Down: Buttered bread is humorously said to always land with the buttered side facing down due to its weight distribut
> The paradox arises from the combination of these two concepts. If the cat is indeed upside down with buttered bread strapped to its back, it creates a hu
> Conclusion:
> In a whimsical sense:
> If you take the paradox literally, one might humorously suggest that neither hits the ground because they enter an eternal spin, or that they both hit at t
> In reality, the outcome would depend on factors like the initial position, how securely the bread is strapped, and how the cat reacts in mid-air.
> Ultimately, this scenario is more about the fun of the paradox than a definitive answer!

Darn it ChatGPT, I want an answer! Personally, my money is on the cat.

Also, I just have to share one of my favorite generations of this prompt, courtesy of Google Vertex AI Gemini:



Figure 2.8: Well played, Google Gemini

Thank you Google Gemini. This looks like me on Monday after I get buttered and smacked down by the bread.

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/image-generation-example

TIP More details on the Image Model API can be found in Spring AI's reference documentation.

Like the Image Model API, we can use Spring AI's Audio Model API to transcribe audio.

Audio model API

Currently, Spring AI supports both Audio Transcription using OpenAI's Whisper model as well as Text-to-Speech using OpenAI's TTS model.

NOTE Audio Transcription is also supported with Azure OpenAI.

The Spring AI API types are specific to OpenAI, so the core interface to use in this case is the `OpenAiAudioTranscriptionModel` class.

Below is a simple Spring AI application using the model to transcribe a pre-generated audio clip.

```
@Bean
ApplicationRunner runner(OpenAiAudioTranscriptionModel model) {
  return args -> {
    Resource audio = new ClassPathResource("/audio.mp3");
    String text = model.call(audio);
    System.out.printf("\"%s\"%n", text.trim());
  };
}
```

Running the program will produce the following output:

```
May the force be with you
```

We will return to the topic of audio in *Chapter 5* on implementing Shazam.

See the complete code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/audio-transcription-example

NOTE The quality of audio transcription will vary by AI provider model and the quality of the audio sample. Several factors may affect transcription, such as pronunciation, noise, distortion, and the quality of recording equipment. In short, mileage varies, and you are encouraged to test with different AI models and settings where applicable.

TIP More details on the Audio Model API (Transcription API) can be found in Spring AI's reference documentation.

Summary

This chapter focused on using Spring AI's core API to interact with different AI models.

Each AI model referenced and discussed had different capabilities and served a different purpose, such as for chat, moderation, similarity search, image generation and audio transcription or **text-to-speech** (**TTS**).

Two of the more common APIs you will frequently use to interact with AI models are Spring AI's `ChatClient` and `Advisors` API. Spring AI's auto-configuration will automatically configure a ChatModel based on the Spring AI Model Starters you decide to declare on your application classpath. However, most of the time you will interface with the AI model indirectly using the higher-order constructs accessible from a `ChatClient`, such as user prompts and `Advisors`.

`Advisors` are particularly useful for encapsulating reusable slices of functionality to further process prompts and AI model responses. We saw that an `Advisor` is precisely how Spring AI implements chat memory, with the `ChatMemoryAdvisor`, which helps an AI model recall things during a conversation, like your name.

Still, a set of `Advisors` might be used to set up a RAG use case, having a single Advisor for each data source from which additional information can be retrieved to augment the prompt. Subsequently, a sequence of downstream `Advisors` can be used to process an AI model's response, such as for auditing purposes and quality controls.

Refer back to this chapter if you ever need a refresher on Spring AI's APIs features based on your functional concerns.

In the next chapter we continue exploring additional capabilities and APIs of Spring AI.

Subscribe for a free eBook.

New frameworks, evolving architectures, research drops, production breakdowns – *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and generative AI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at https://packt.link/8Oz6Y or scan the QR code below.



Previous chapter
‹ Chapter 1: Getting Started with Spring AI

Next chapter
Chapter 3: Understanding Spring AI: Part 2 ›